

Laporan Praktikum Pekan 6

“Decision Tree (Overfitting & Tree Depth)”

Disusun Untuk Memenuhi Tugas Mata Kuliah MACHINE LEARNING

DOSEN PENGAMPU:

Derisma, M.T.



DISUSUN OLEH:

Karimah Irsyadiyah (2411533018)

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
T.A 2025/2026**

Daftar Isi

BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan Praktikum.....	3
LINK GITHUB:.....	3
BAB II LANGKAH-LANGKAH PRAKTIKUM.....	4
File DecisionTree.ipynb.....	4
2.1 Import Library.....	4
2.2 Load Dataset	4
2.3 Decision Tree Default	5
2.4 Uji Berbagai Nilai max_depth	6
2.5 Perbandingan Kriteria Split: Gini vs Entropy	7
2.6 Visualisasi Tree	7
2.7 Tabel Perbandingan Hasil	8
TUGAS PRAKTIKUM	10
Tugas_Decision_Tree.ipynb	10
1. Uji max_depth Hingga 30.....	10
2. Menambahkan min_samples_split.....	12
3. Menambahkan min_samples_leaf.....	13
4. Perbandingan dengan Logistic Regression	14
5. 5-Fold Cross Validation	15
6. Tabel Hasil Tugas Individu	16
PERTANYAAN REFLEKTIF	17
1. Mengapa Decision Tree mudah overfit?	17
2. Mengapa model tanpa batas depth hampir selalu train accuracy = 1?.....	17
3. Mengapa Random Forest bisa mengurangi overfitting?	17
4. Kapan Decision Tree lebih baik dari Logistic Regression?	17
5. Apakah model paling kompleks selalu terbaik?	17
BAB III PENUTUP	18
3.1 Kesimpulan	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Decision Tree adalah salah satu algoritma machine learning yang digunakan untuk melakukan klasifikasi maupun regresi dengan cara membagi data berdasarkan fitur yang paling informatif. Algoritma ini membentuk struktur pohon keputusan yang terdiri dari root node, decision node, dan leaf node. Setiap percabangan pada pohon merepresentasikan proses pemilihan fitur terbaik untuk memisahkan data ke dalam kelas tertentu.

Pada praktikum ini, Decision Tree diterapkan pada dataset Breast Cancer untuk mengklasifikasikan data menjadi dua kelas. Praktikum ini bertujuan untuk memahami cara kerja Decision Tree, menganalisis pengaruh parameter `max_depth` terhadap performa model, serta mengamati gejala overfitting dan underfitting. Selain itu, dilakukan perbandingan antara kriteria split Gini dan Entropy, serta visualisasi struktur pohon keputusan agar proses pemisahan data dapat dipahami dengan lebih jelas.

1.2 Tujuan Praktikum

1. Memahami cara kerja Decision Tree.
2. Menjelaskan konsep Gini dan Entropy.
3. Menganalisis pengaruh `max_depth` terhadap performa model.
4. Mengidentifikasi overfitting dan underfitting.
5. Membandingkan performa Decision Tree dengan model lain.
6. Memvisualisasikan struktur pohon keputusan.

LINK GITHUB:

https://github.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018_ML2526/tree/main/PRAKTIKUM%206

BAB II

LANGKAH-LANGKAH PRAKTIKUM

File DecisionTree.ipynb

2.1 Import Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.tree import plot_tree
```

Penjelasan:

Pada tahap awal dilakukan import library yang diperlukan untuk pengolahan data, visualisasi, pelatihan model, dan evaluasi performa model. Library numpy, pandas, dan matplotlib digunakan untuk manipulasi data dan visualisasi. Sementara itu, load_breast_cancer digunakan untuk memuat dataset Breast Cancer, DecisionTreeClassifier digunakan untuk membangun model Decision Tree, dan accuracy_score digunakan untuk menghitung akurasi model. Fungsi plot_tree digunakan untuk menampilkan visualisasi pohon keputusan.

2.2 Load Dataset

```
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Penjelasan:

Dataset Breast Cancer dimuat menggunakan fungsi load_breast_cancer(). Dataset kemudian dipisahkan menjadi variabel fitur X dan target y. Selanjutnya, data dibagi

menjadi data latih dan data uji menggunakan `train_test_split()` dengan proporsi 80% data latih dan 20% data uji. Pembagian ini dilakukan agar model dapat dilatih pada sebagian data dan diuji pada data yang belum pernah dilihat sebelumnya.

2.3 Decision Tree Default

```
tree_default = DecisionTreeClassifier(random_state=42)
tree_default.fit(X_train, y_train)

y_train_pred = tree_default.predict(X_train)
y_test_pred = tree_default.predict(X_test)

print("Train Accuracy: ", accuracy_score(y_train, y_train_pred))
print("Test Accuracy: ", accuracy_score(y_test, y_test_pred))
```

... Train Accuracy: 1.0
Test Accuracy: 0.9473684210526315

Penjelasan:

Pada tahap ini dibuat model Decision Tree tanpa batas kedalaman (`max_depth` default). Model dilatih menggunakan data training dan kemudian dilakukan prediksi pada data training dan testing. Nilai akurasi dihitung untuk melihat performa model pada kedua data tersebut.

Analisis:

Pada Decision Tree tanpa pembatasan kedalaman, model cenderung sangat mudah menyesuaikan diri terhadap data training. Jika train accuracy mendekati 1.0 dan test accuracy jauh lebih rendah, maka hal ini menunjukkan adanya overfitting. Sebaliknya, jika perbedaannya tidak terlalu besar, model masih memiliki kemampuan generalisasi yang cukup baik

2.4 Uji Berbagai Nilai max_depth

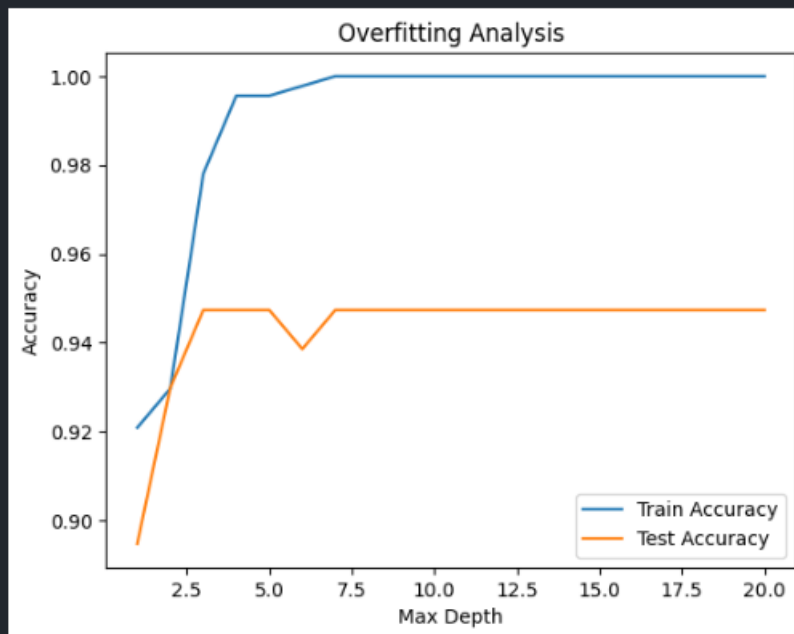
```
depths = range(1,21)

train_acc = []
test_acc = []

for d in depths:
    model = DecisionTreeClassifier(max_depth=d, random_state=42)
    model.fit(X_train, y_train)

    train_acc.append(accuracy_score(y_train, model.predict(X_train)))
    test_acc.append(accuracy_score(y_test, model.predict(X_test)))

plt.plot(depths, train_acc, label="Train Accuracy")
plt.plot(depths, test_acc, label='Test Accuracy')
plt.xlabel("Max Depth")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Overfitting Analysis")
plt.show()
```



Penjelasan:

Pada tahap ini dilakukan eksperimen menggunakan berbagai nilai max_depth dari 1 sampai 20. Tujuannya adalah untuk melihat bagaimana kedalaman pohon memengaruhi akurasi model pada data training dan testing. Hasil akurasi kemudian divisualisasikan dalam grafik untuk memudahkan analisis overfitting.

Analisis:

Semakin besar nilai max_depth, model menjadi semakin kompleks dan cenderung mampu menyesuaikan diri dengan data training. Jika train accuracy terus meningkat hingga sangat

tinggi, sementara test accuracy justru stagnan atau menurun setelah titik tertentu, maka model mulai mengalami overfitting. Nilai depth terbaik biasanya berada pada titik saat test accuracy paling tinggi sebelum model menjadi terlalu kompleks.

2.5 Perbandingan Kriteria Split: Gini vs Entropy

```
tree_gini = DecisionTreeClassifier(criterion="gini", max_depth=5, random_state=42)
tree_entropy = DecisionTreeClassifier(criterion="entropy", max_depth=5, random_state=42)

tree_gini.fit(X_train, y_train)
tree_entropy.fit(X_train, y_train)

acc_gini = accuracy_score(y_test, tree_gini.predict(X_test))
acc_entropy = accuracy_score(y_test, tree_entropy.predict(X_test))

print("Accuracy (Gini) : ", acc_gini)
print("Accuracy (Entropy) : ", acc_entropy)
```

Accuracy (Gini) : 0.9473684210526315
Accuracy (Entropy) : 0.9473684210526315

Penjelasan:

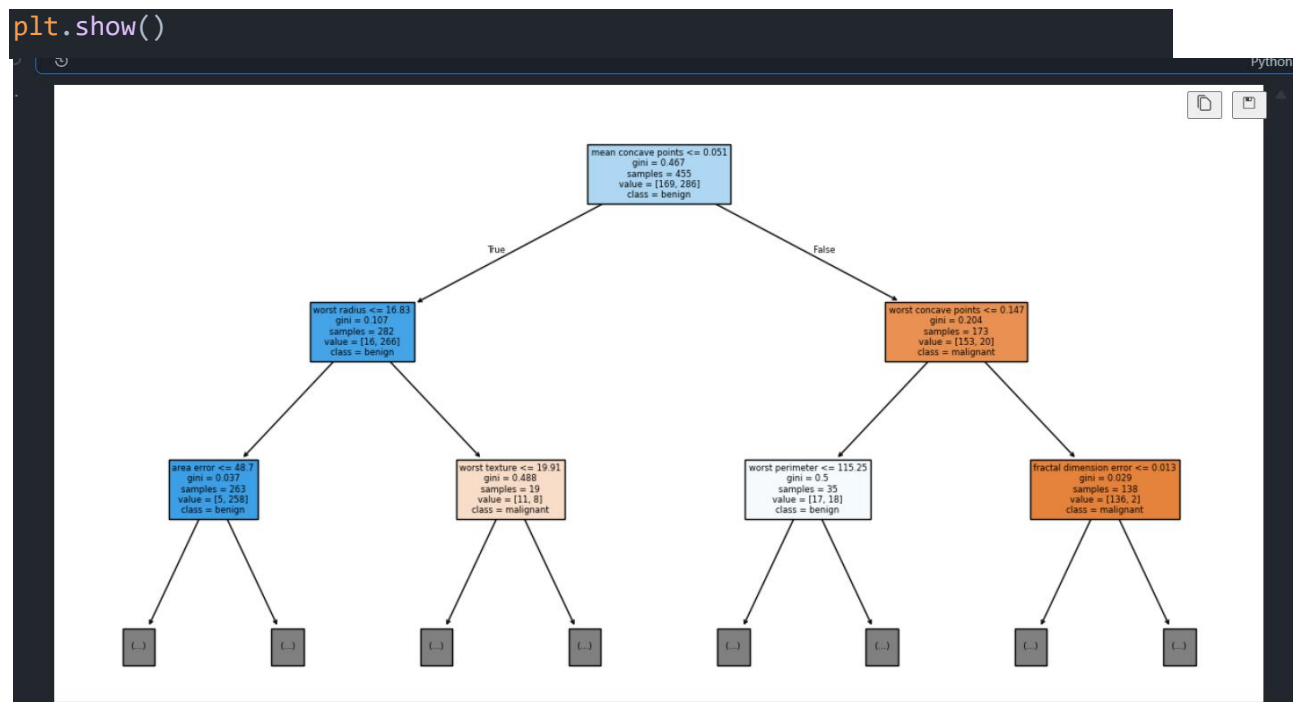
Pada tahap ini dilakukan perbandingan antara dua kriteria split yang umum digunakan pada Decision Tree, yaitu Gini dan Entropy. Kedua model dilatih menggunakan `max_depth=5` agar perbandingan menjadi lebih adil. Setelah itu, akurasi masing-masing model dihitung pada data testing.

Analisis:

Gini dan Entropy sama-sama digunakan untuk mengukur impurity atau ketidakmurnian data pada node. Perbedaannya biasanya tidak terlalu besar dalam praktik. Jika hasil accuracy keduanya hampir sama, maka dapat disimpulkan bahwa pada dataset ini kedua kriteria split memberikan performa yang relatif setara. Pemilihan di antara keduanya biasanya bergantung pada hasil eksperimen dan kebutuhan model.

2.6 Visualisasi Tree

```
plt.figure(figsize=(15,8))
plot_tree(
    tree_gini,
    feature_names=X.columns,
    class_names=data.target_names,
    filled=True,
    max_depth=2
)
```



Penjelasan:

Visualisasi pohon keputusan dilakukan untuk melihat struktur model Decision Tree secara langsung. Pada visualisasi ini, hanya ditampilkan hingga kedalaman 2 agar pohon lebih mudah dibaca. Setiap node menampilkan fitur yang digunakan untuk split, nilai impurity, jumlah sampel, dan kelas dominan pada node tersebut.

Diskusi:

Fitur yang menjadi root node adalah fitur yang dianggap paling penting oleh model dalam membedakan kelas. Fitur ini dipilih karena mampu memberikan pemisahan terbaik berdasarkan kriteria Gini atau Entropy. Untuk laporan, nama fitur root node perlu disesuaikan dengan hasil visualisasi pada notebook kamu.

2.7 Tabel Perbandingan Hasil

```

results_depth = pd.DataFrame({
    'Depth': list(depths),
    'Train Accuracy': np.round(train_acc, 4),
    'Test Accuracy': np.round(test_acc, 4)
})

results_depth

```

[8] ✓ 0.0s

	Depth	Train Accuracy	Test Accuracy
0	1	0.9209	0.8947
1	2	0.9297	0.9298
2	3	0.9780	0.9474
3	4	0.9956	0.9474
4	5	0.9956	0.9474
5	6	0.9978	0.9386
6	7	1.0000	0.9474
7	8	1.0000	0.9474
8	9	1.0000	0.9474
9	10	1.0000	0.9474
10	11	1.0000	0.9474
11	12	1.0000	0.9474
12	13	1.0000	0.9474
13	14	1.0000	0.9474
14	15	1.0000	0.9474
15	16	1.0000	0.9474
16	17	1.0000	0.9474
17	18	1.0000	0.9474
18	19	1.0000	0.9474
19	20	1.0000	0.9474

Penjelasan:

Tabel ini digunakan untuk membandingkan performa model pada berbagai nilai `max_depth`. Dari tabel tersebut dapat dilihat bagaimana perubahan kedalaman pohon memengaruhi akurasi training dan testing. Tabel ini juga membantu menentukan kedalaman optimal yang menghasilkan keseimbangan terbaik antara kompleksitas model dan kemampuan generalisasi.

TUGAS PRAKTIKUM

Tugas_Decision_Tree.ipynb

1. Uji max_depth Hingga 30

```
## Uji max_depth sampai 30
depths = range(1, 31)

train_acc_30 = []
test_acc_30 = []

for d in depths:
    model = DecisionTreeClassifier(
        max_depth=d,
        random_state=42
    )

    model.fit(X_train, y_train)

    train_acc_30.append(
        accuracy_score(y_train, model.predict(X_train))
    )

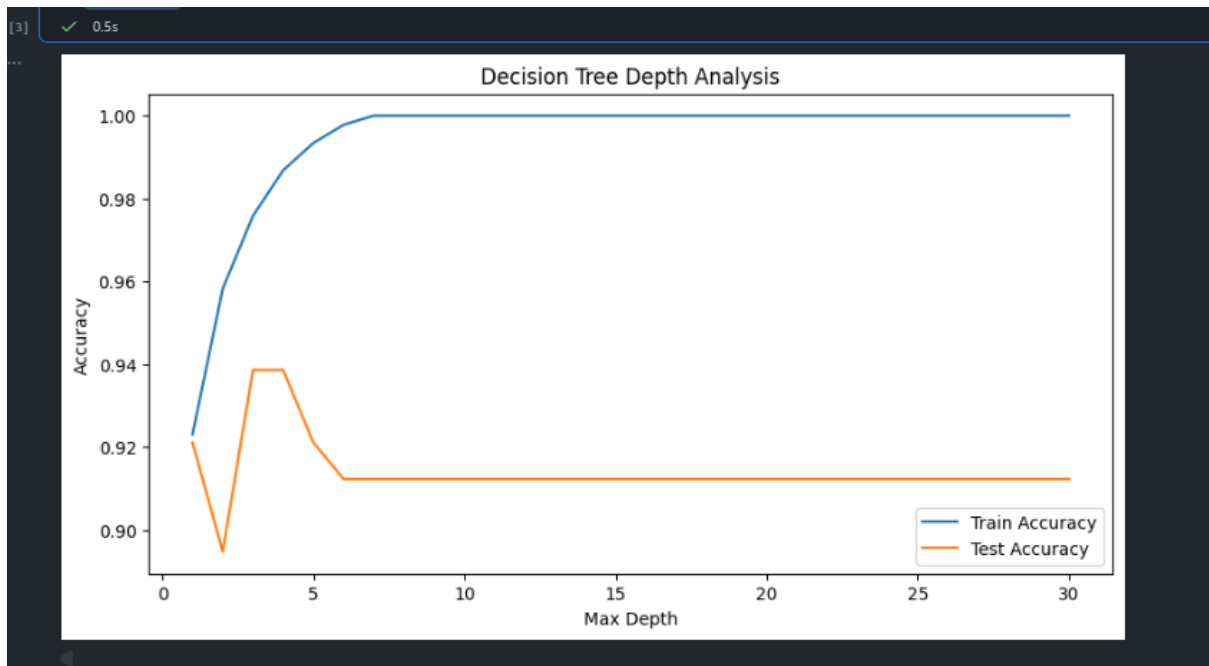
    test_acc_30.append(
        accuracy_score(y_test, model.predict(X_test))
    )

plt.figure(figsize=(10,5))

plt.plot(depths, train_acc_30, label='Train Accuracy')
plt.plot(depths, test_acc_30, label='Test Accuracy')

plt.xlabel("Max Depth")
plt.ylabel("Accuracy")
plt.title("Decision Tree Depth Analysis")
plt.legend()

plt.show()
```



Penjelasan:

Kode di atas digunakan untuk menguji pengaruh parameter `max_depth` dari 1 hingga 30 terhadap performa model Decision Tree.

Pada setiap nilai `depth`, model dilatih menggunakan data training kemudian dihitung nilai accuracy pada data training dan testing. Hasil accuracy disimpan ke dalam list `train_acc_30` dan `test_acc_30`.

Grafik digunakan untuk melihat hubungan antara kompleksitas model dengan performa klasifikasi. Semakin besar nilai `max_depth`, model menjadi lebih kompleks dan cenderung lebih mudah mengalami overfitting.

Analisis:

Berdasarkan grafik, train accuracy terus meningkat seiring bertambahnya nilai `max_depth`. Pada `depth` tertentu, train accuracy bahkan dapat mencapai 1.0 yang menunjukkan bahwa model sangat mampu menghafal data training.

Namun, test accuracy tidak selalu meningkat. Setelah mencapai titik optimal, test accuracy cenderung stagnan atau menurun. Hal ini menunjukkan bahwa model mulai mengalami overfitting karena terlalu kompleks dan kurang mampu melakukan generalisasi terhadap data baru.

2. Menambahkan min_samples_split

```
## Tambahkan min_samples_split
split_values = [2, 5, 10, 20, 30]
split_results = []

for s in split_values:
    model = DecisionTreeClassifier(
        max_depth=5,
        min_samples_split=s,
        random_state=42
    )
    model.fit(X_train, y_train)

    train_acc_split = accuracy_score(y_train, model.predict(X_train))
    test_acc_split = accuracy_score(y_test, model.predict(X_test))

    split_results.append([s, train_acc_split, test_acc_split])
    print(f"min_samples_split={s} -> Train Acc={train_acc_split:.4f}, Test Acc={test_acc_split:.4f}")

split_df = pd.DataFrame(
    split_results,
    columns=["min_samples_split", "Train Accuracy", "Test Accuracy"]
)

split_df
```

✓ 0.0s

min_samples_split=2 -> Train Acc=0.9934, Test Acc=0.9211
min_samples_split=5 -> Train Acc=0.9912, Test Acc=0.9211
min_samples_split=10 -> Train Acc=0.9868, Test Acc=0.9123
min_samples_split=20 -> Train Acc=0.9780, Test Acc=0.9386
min_samples_split=30 -> Train Acc=0.9758, Test Acc=0.9386

	min_samples_split	Train Accuracy	Test Accuracy
0	2	0.993407	0.921053
1	5	0.991209	0.921053
2	10	0.986813	0.912281
3	20	0.978022	0.938596
4	30	0.975824	0.938596

Penjelasan

Parameter `min_samples_split` menentukan jumlah minimum sampel yang diperlukan agar sebuah node dapat dibagi kembali.

Semakin besar nilai parameter ini, model menjadi lebih sederhana karena proses split pada pohon dibatasi.

Analisis

Jika nilai `min_samples_split` terlalu kecil, pohon keputusan menjadi sangat kompleks dan berpotensi mengalami overfitting. Sebaliknya, jika nilainya terlalu besar, model menjadi terlalu sederhana sehingga dapat menyebabkan underfitting.

Nilai parameter yang optimal biasanya menghasilkan keseimbangan antara kompleksitas model dan kemampuan generalisasi.

3. Menambahkan min_samples_leaf

```
## Tambahkan min_samples_leaf
leaf_values = [1, 2, 5, 10, 20]
leaf_results = []

for l in leaf_values:
    model = DecisionTreeClassifier(
        max_depth=5,
        min_samples_leaf=l,
        random_state=42
    )
    model.fit(X_train, y_train)

    train_acc_leaf = accuracy_score(y_train, model.predict(X_train))
    test_acc_leaf = accuracy_score(y_test, model.predict(X_test))

    leaf_results.append([l, train_acc_leaf, test_acc_leaf])
    print(f"min_samples_leaf={l} -> Train Acc={train_acc_leaf:.4f}, Test Acc={test_acc_leaf:.4f}")

leaf_df = pd.DataFrame(
    leaf_results,
    columns=["min_samples_leaf", "Train Accuracy", "Test Accuracy"]
)

leaf_df
```

✓ 0.1s

min_samples_leaf=1 -> Train Acc=0.9934, Test Acc=0.9211
min_samples_leaf=2 -> Train Acc=0.9912, Test Acc=0.9035
min_samples_leaf=5 -> Train Acc=0.9758, Test Acc=0.9211
min_samples_leaf=10 -> Train Acc=0.9626, Test Acc=0.9474
min_samples_leaf=20 -> Train Acc=0.9473, Test Acc=0.9035

	min_samples_leaf	Train Accuracy	Test Accuracy
0	1	0.993407	0.921053
1	2	0.991209	0.903509
2	5	0.975824	0.921053
3	10	0.962637	0.947368
4	20	0.947253	0.903509

Penjelasan

Parameter `min_samples_leaf` menentukan jumlah minimum sampel yang harus dimiliki setiap leaf node.

Parameter ini membantu mengurangi pembentukan leaf node yang terlalu kecil sehingga model menjadi lebih stabil.

Analisis

Semakin besar nilai `min_samples_leaf`, struktur pohon menjadi lebih sederhana dan risiko overfitting berkurang. Namun jika nilainya terlalu besar, model dapat kehilangan kemampuan untuk menangkap pola penting pada data.

4. Perbandingan dengan Logistic Regression

```
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(max_iter=5000)

logreg.fit(X_train, y_train)

y_pred_logreg = logreg.predict(X_test)

logreg_acc = accuracy_score(y_test, y_pred_logreg)

tree_model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

tree_model.fit(X_train, y_train)

tree_acc = accuracy_score(
    y_test,
    tree_model.predict(X_test)
)

print("Logistic Regression Accuracy :", logreg_acc)
print("Decision Tree Accuracy       :", tree_acc)
```

✓ 0.4s

Logistic Regression Accuracy : 0.9649122807017544
Decision Tree Accuracy : 0.9210526315789473

Penjelasan

Pada tahap ini dilakukan perbandingan antara model Decision Tree dan Logistic Regression menggunakan dataset yang sama.

Tujuannya adalah untuk melihat perbedaan performa antara model linear dan model berbasis pohon keputusan.

Analisis

Decision Tree mampu membentuk batas keputusan yang lebih fleksibel dibanding Logistic Regression karena tidak terbatas pada hubungan linear. Namun, Logistic Regression biasanya lebih stabil dan lebih tahan terhadap overfitting.

Jika hasil accuracy keduanya mirip, maka Logistic Regression dapat menjadi pilihan yang lebih sederhana dan efisien.

5. 5-Fold Cross Validation

```
## 5-Fold Cross Validation untuk Decision Tree
from sklearn.model_selection import cross_val_score

cv_model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

cv_scores = cross_val_score(
    cv_model,
    X,
    y,
    cv=5,
    scoring='accuracy'
)

print("Cross Validation Scores:")
print(cv_scores)

print("Mean Accuracy:", cv_scores.mean())
```

✓ 0.0s

Cross Validation Scores:
[0.89473684 0.9122807 0.92982456 0.94736842 0.91150442]
Mean Accuracy: 0.91914299021891

Penjelasan

5-Fold Cross Validation digunakan untuk mengevaluasi performa model secara lebih stabil dan objektif.

Dataset dibagi menjadi 5 bagian, kemudian model dilatih dan diuji secara bergantian pada setiap fold.

Analisis

Cross validation membantu mengurangi bias akibat pembagian data train-test tertentu. Jika hasil accuracy antar fold relatif konsisten, maka model memiliki performa yang stabil dan mampu melakukan generalisasi dengan baik.

Sebaliknya, jika variasi accuracy antar fold sangat besar, maka model kurang stabil dan kemungkinan sensitif terhadap perubahan data training.

6. Tabel Hasil Tugas Individu

```
# Tabel Perbandingan Akhir
hasil_tugas = pd.DataFrame({
    'Model': [
        'Decision Tree',
        'Logistic Regression'
    ],
    'Accuracy': [
        tree_acc,
        logreg_acc
    ]
})

hasil_tugas
```

✓ 0.0s

	Model	Accuracy
0	Decision Tree	0.921053
1	Logistic Regression	0.964912

Penjelasan

Tabel digunakan untuk membandingkan performa model Decision Tree dan Logistic Regression berdasarkan nilai accuracy yang diperoleh pada data testing.

PERTANYAAN REFLEKTIF

1. Mengapa Decision Tree mudah overfit?

Decision Tree mudah mengalami overfitting karena model ini sangat fleksibel dalam mempelajari pola data training. Jika pohon dibiarkan tumbuh terlalu dalam, model akan terus membagi data hingga mampu menghafal hampir seluruh data training, termasuk noise dan pola yang sebenarnya tidak penting. Akibatnya, performa pada data training menjadi sangat tinggi, tetapi kemampuan generalisasi terhadap data baru menjadi menurun.

2. Mengapa model tanpa batas depth hampir selalu train accuracy = 1?

Model tanpa batas depth dapat terus melakukan splitting hingga setiap leaf node berisi sangat sedikit data atau bahkan satu data saja. Hal ini membuat model mampu menghafal seluruh data training dengan sangat detail sehingga train accuracy sering mencapai 1.0. Namun, kondisi ini biasanya menyebabkan overfitting karena model terlalu fokus pada data training.

3. Mengapa Random Forest bisa mengurangi overfitting?

Random Forest mengurangi overfitting dengan cara menggabungkan banyak Decision Tree yang dilatih menggunakan subset data dan subset fitur yang berbeda-beda. Hasil prediksi akhir diperoleh dari voting seluruh tree sehingga model menjadi lebih stabil dan tidak terlalu bergantung pada satu pohon keputusan saja. Teknik ini membantu mengurangi variance dan meningkatkan kemampuan generalisasi model.

4. Kapan Decision Tree lebih baik dari Logistic Regression?

Decision Tree lebih baik digunakan ketika hubungan antar fitur dan target bersifat non-linear atau memiliki pola yang kompleks. Selain itu, Decision Tree juga lebih mudah diinterpretasikan karena menghasilkan struktur aturan berbentuk pohon keputusan. Sebaliknya, Logistic Regression lebih cocok digunakan ketika hubungan data cenderung linear dan membutuhkan model yang lebih sederhana serta stabil.

5. Apakah model paling kompleks selalu terbaik?

Tidak. Boundary yang terlalu kompleks dapat menyebabkan overfitting karena model terlalu mengikuti pola data training dan kurang mampu melakukan generalisasi pada data baru.

BAB III

PENUTUP

3.1 Kesimpulan

Pada Praktikum 6 ini telah dilakukan implementasi algoritma Decision Tree menggunakan dataset Breast Cancer untuk tugas klasifikasi. Hasil praktikum menunjukkan bahwa Decision Tree mampu menghasilkan performa klasifikasi yang baik dan mudah dipahami melalui visualisasi struktur pohon keputusan.

Pengujian terhadap berbagai nilai `max_depth` menunjukkan bahwa kedalaman pohon sangat memengaruhi performa model. Semakin besar nilai `depth`, model menjadi semakin kompleks dan `train accuracy` cenderung meningkat hingga mendekati 1.0. Namun, model yang terlalu kompleks dapat menyebabkan `overfitting` karena performa pada data testing tidak selalu meningkat.

Perbandingan antara kriteria Gini dan Entropy menunjukkan bahwa keduanya menghasilkan performa yang relatif mirip pada dataset Breast Cancer. Selain itu, penggunaan parameter seperti `min_samples_split` dan `min_samples_leaf` membantu mengontrol kompleksitas model agar lebih stabil.

Berdasarkan hasil perbandingan dengan Logistic Regression dan evaluasi menggunakan 5-Fold Cross Validation, dapat disimpulkan bahwa Decision Tree memiliki keunggulan dalam menangani pola non-linear dan mudah divisualisasikan, namun tetap memerlukan pengaturan parameter yang tepat agar tidak mengalami `overfitting`. Secara keseluruhan, praktikum ini memberikan pemahaman mengenai konsep Decision Tree, `overfitting`, tuning parameter, dan evaluasi performa model klasifikasi.